

Dokumentasi Kode Simulasi NMPC Konvensional Tanpa Halangan

1. Pendefinisian Parameter Fisik Kapal

```
clc; clear; close all;

%% Parameter Kapal
L = 101.07;          % Panjang Kapal (m)
B = 14;              % Lebar Kapal (m)
T = 3.7;             % Draft Kapal (m)
m = 2423*1e3;        % Massa Kapal (kg)
u_0 = 15.4;          % Kecepatan surge Kapal (m/s)
C_B = 0.65;          % Koefisien Blok
x_G = 5.25;          % Pusat massa sumbu-x
A_delta = 5.7224;    % Luas rudder (m^2)
rho = 1024;           % Massa jenis air laut (kg/m^3)
r = 0.156*L;         % Kisaran jari-jari girasa
```

2. Perhitungan Koefisien Hidrodinamika

```
%% Koefisien Hidrodinamika
Y_v_dot = -(1 + 0.16*C_B*B/T - 5.1*(B/L)^2)*pi*(T/L)^2;
Y_r_dot = -(0.67*(B/L) - 0.0033*(B/T)^2)*pi*(T/L)^2;
N_v_dot = -(1.1*B/L - 0.041*B/T)*pi*(T/L)^2;
N_r_dot = -((1/12) + 0.017*C_B*B/T - 0.33*B/L)*pi*(T/L)^2;
Y_v = -(1 + 0.4*C_B*B/T)*pi*(T/L)^2;
Y_r = -(-0.5 + 2.2*B/L - 0.08*B/T)*pi*(T/L)^2;
N_v = -(0.5 + 2.4*T/L)*pi*(T/L)^2;
N_r = -(0.25 + 0.039*B/T - 0.56*B/L)*pi*(T/L)^2;

m_nd = 2*m/(rho*L^3);
x_G_nd = x_G/L;
I_z_nd = 1.2392*10^(-4);
u_0_nd = 1;
```

3. Pembentukan Model Gerak Kapal

```
%% Model Gerak Kapal
M_mat = [m_nd-Y_v_dot , m_nd*x_G_nd-Y_r_dot ;
         m_nd*x_G_nd - N_v_dot , I_z_nd - N_r_dot ];

a11 = ((I_z_nd - N_r_dot)*Y_v - ...
        (m_nd*x_G_nd - Y_r_dot)*N_v)/det(M_mat);
a12 = ((I_z_nd - N_r_dot)*(Y_r - m_nd*u_0_nd) - ...
        (m_nd*x_G_nd - Y_r_dot)*(N_r - m_nd*x_G_nd*u_0_nd))/det(M_mat);
a21 = ((m_nd - Y_v_dot)*N_v - ...
        (m_nd*x_G_nd - N_v_dot)*Y_v)/det(M_mat);
a22 = ((m_nd - Y_v_dot)*(N_r - m_nd*x_G_nd*u_0_nd) - ...
        (m_nd*x_G_nd - N_v_dot)*(Y_r - m_nd*u_0_nd))/det(M_mat);

A_sys = [a11 , a12 ;
         a21 , a22 ];
B_sys = [0.01 ;
         1];
```

4. Setup Parameter NMPC Konvensional

Pada NMPC konvensional, nilai N (jumlah langkah horizon prediksi) diturunkan langsung dari pembagian horizon waktu T_p dengan langkah waktu sampling T_{sim} , bukan dipilih sebagai orde polinomial kolokasi.

```
% Setup NMPC
Tp = 30;           % Horizon waktu prediksi (s)
T_sim = 1;         % Langkah waktu simulasi / interval waktu simulasi (s)
T_sim_total = 150; % Waktu simulasi total (s)
N = round(Tp / T_sim); % Banyak titik prediksi secara diskrit
```

Dengan $T_p = 30$ detik dan $T_{sim} = 1$ detik, maka $N = 30$ langkah prediksi. Artinya, solver akan merencanakan serangkaian 30 nilai sudut kemudi sekaligus untuk 30 detik ke depan.

4.1. Definisi Matriks Pembobot Fungsi Objektif

Didefinisikan matriks pembobot Q dan R . Matriks Q menentukan besarnya biaya yang dikenakan terhadap kesalahan pelacakan, sedangkan R menentukan besarnya biaya penggunaan kontrol.

```
Q = diag([10, 1, 1]); % Bobot state
R = 1;                 % Bobot input
```

4.2. Penentuan Batasan Fisik Kapal dan Aktuator

Didefinisikan batasan fisik (*constraints*) gerakan kapal dan kemudi. Kecepatan putar sudut haluan kapal (*yaw rate*) dibatasi sebesar ± 0.0932 rad/s (r_limit), maksimal sudut *rudder* sebesar ± 35 derajat (u_limit), dan maksimal perubahan sudut *rudder* sebesar ± 5 derajat per detik (u_rate_limit). Untuk batasan sudut *rudder* dan perubahan sudut *rudder* dilakukan konversi satuan dari derajat ke radian menggunakan perintah `deg2rad`.

```
% Batasan state (yaw rate)
r_limit = 0.0932; % (dalam radian)
r_limit_nd = r_limit * (L / u_0);

% Batasan input (rudder)
u_limit = deg2rad(35);

% Batasan perubahan input (rudder)
u_rate_limit = deg2rad(5);
du_max = u_rate_limit * T_sim;

u_prev = 0; % Inisialisasi input awal
```

Variabel du_max adalah batas maksimum perubahan sudut kemudi yang diizinkan dalam *satu langkah waktu* T_{sim} . Nilai ini diperoleh dengan mengalikan laju perubahan maksimum (u_rate_limit) dengan panjang satu langkah waktu (T_{sim}): $\Delta u_{max} = 5^\circ \times 1 \text{ s} = 5^\circ$ per langkah. Variabel u_prev menyimpan nilai sudut kemudi pada langkah waktu sebelumnya, diinisialisasi ke 0 (kemudi lurus).

4.3. Pembentukan Kendala Linear Perubahan Input

Pada NMPC konvensional, kendala perubahan sudut kemudi ($|\Delta u| \leq \Delta u_{max}$) *dirumuskan terlebih dahulu sebagai matriks kendala linear* sebelum loop simulasi berjalan, lalu langsung diumpankan ke `fmincon` sebagai argumen A_{du} dan b_{du} .

```
% Batasan Perubahan Input
[A_du, b_du] = du_constraints(N, u_prev, du_max);
```

Pemanggilan fungsi `du_constraints` ini menghasilkan sepasang matriks (A_{du} , b_{du}) yang menyandikan seluruh rantai batasan perubahan kemudi dalam bentuk sistem pertidaksamaan linear $A_{du} \cdot U \leq b_{du}$.

4.4. Penggabungan Batas Atas dan Batas Bawah Input

Pada NMPC konvensional, variabel keputusan yang diserahkan ke `fmincon` hanyalah deretan nilai input kemudi $U = [u_1, u_2, \dots, u_N]^T$ (sebanyak N elemen). Karena itu, batas atas (`ub`) dan batas bawah (`lb`) cukup berupa vektor N elemen yang semuanya bernilai sama $\pm 35^\circ$.

```
% Batasan Batas Atas dan Batas Bawah Input
lb = -u_limit * ones(N,1);
ub = u_limit * ones(N,1);
```

5. Inisialisasi Kondisi Awal Simulasi

```
s0_dim = [0; 0; 0; 100; 0]; % Inisialisasi kondisi awal [v; r; x; y; psi]
s0_nd = dimensional_to_nondimensional(s0_dim, L, u_0); % Konversi ke nondimensional

h_ref = [0; 0; 0]; % Lintasan Referensi

history_state_nd = s0_nd; % Inisialisasi riwayat state
history_input = []; % Inisialisasi riwayat input
s_nd = s0_nd;
```

Vektor `s0_dim` berisi lima elemen kondisi awal kapal dalam satuan dimensional: kecepatan *sway* v (m/s), kecepatan putar haluan *yaw rate* r (rad/s), posisi x (m), posisi y (m), dan sudut haluan ψ (rad). Nilai $y = 100$ m berarti kapal dimulai sejauh 100 meter dari garis referensi, sehingga pengendali harus segera melakukan koreksi untuk mengembalikan kapal ke lintasan referensi $y = 0$.

Variabel `s_nd` adalah salinan state kapal yang akan diperbarui di setiap iterasi loop dan digunakan sebagai titik awal perhitungan di iterasi berikutnya.

5.1. Pengaturan Konfigurasi Solver Optimasi

```
% options optimizer
options = optimoptions('fmincon', 'Algorithm', 'sqp', 'Display', 'none', ...
    'MaxIterations', 200, 'OptimalityTolerance', 1e-6);
```

'MaxIterations', 200 membatasi jumlah iterasi pencarian solusi. 'OptimalityTolerance', $1e-6$ menetapkan ambang batas kepresisian gradien fungsi tujuan. Solver akan berhenti jika gradien sudah cukup kecil, menandakan solusi sudah mendekati optimal.

5.2. Pemicuan Timer Komputasi

```
fprintf('Memulai simulasi NMPC \n');
timer_val = tic; % Memulai timer
```

Perintah `tic` berfungsi menyalakan sistem pewaktu internal komputer (ibarat memicu tombol *start* pada *stopwatch*) menandakan dimulainya pencatatan waktu kalkulasi proses simulasi secara keseluruhan.

6. Loop Utama Simulasi NMPC

Program berjalan dengan langkah sebesar `T_sim` detik. Batas atas loop adalah `T_sim_total - T_sim` (bukan `T_sim_total`) karena setiap iterasi sudah menghitung *satu langkah ke depan*, sehingga iterasi terakhir pada $t = T_{\text{sim_total}} - T_{\text{sim}}$ menghasilkan state pada $t = T_{\text{sim_total}}$.

```
for t = 0:T_sim:T_sim_total-T_sim
```

6.1. Inisialisasi Lintasan Referensi untuk Horizon Prediksi

```
% Referensi untuk horizon prediksi
t_pred = t + (1:N)*T_sim;
x_ref_seq = (h_ref(1) + t_pred * u_0) / L; % x_ref nondimensional
y_ref_seq = h_ref(2) / L * ones(N,1); % y_ref nondimensional
psi_ref_seq = h_ref(3) * ones(N,1); % psi_ref nondimensional
```

Vektor $\mathbf{t_pred}$ berisi N titik waktu masa depan yang tersebar seragam: $[t + 1, t + 2, \dots, t + N]$ detik. Lintasan referensi x dihitung dengan mengasumsikan kapal bergerak maju lurus dengan kecepatan tetap u_0 : posisi referensi x pada waktu t_{pred} adalah $x_{\text{ref}} = (x_{\text{ref},0} + t_{\text{pred}} \cdot u_0)/L$ (dalam skala non-dimensional). Nilai y dan ψ referensi bersifat konstan sepanjang horizon.

6.2. Formulasi Fungsi Tujuan dan Kendala

```
% fungsi objektif
cost_fun = @(U) mpc_cost(U, s_nd, u_prev, x_ref_seq, y_ref_seq, psi_ref_seq, ...
    T_sim, L, u_0, A_sys, B_sys, u_0_nd, Q, R);

% fungsi kendala
nonlcon = @(U) state_constraints(U, s_nd, T_sim, L, u_0, A_sys, B_sys, u_0_nd, r_limit_nd);
```

Pada NMPC konvensional, fungsi biaya (`cost_fun`) dan fungsi kendala nonlinear (`nonlcon`) hanya menerima deretan nilai input U sebagai variabel keputusan. Kondisi awal kapal saat ini (`s_nd`) dimasukkan sebagai parameter tetap yang menentukan titik awal simulasi *ke depan* di dalam setiap fungsi. Ini adalah ciri khas pendekatan *sequential shooting*: state kapal tidak dioptimasi secara langsung, melainkan dipropagasi secara berurutan dari kondisi awal menggunakan metode integrasi numerik (Euler).

6.3. Tebakan Awal dan Panggilan Solver Optimasi

```
% Input sebelumnya dalam bentuk matriks sebanyak N×1
U0 = u_prev * ones(N,1);

% Optimasi
% Hasil dari optimasi adalah berupa u optimal (sudut rudder)
[U_opt, ~, exitflag] = fmincon(cost_fun, U0, A_du, b_du, [], [], lb, ub, nonlcon, options);

% Mengecek apakah optimasi berhasil
if exitflag <= 0
    warning('fmincon gagal pada t=%.1f s, exitflag=%d', t, exitflag);
    U_opt = U0;
end
```

Tebakan awal U_0 berisi N nilai yang semuanya sama dengan sudut kemudi langkah waktu sebelumnya (`u_prev`). Strategi ini lebih sederhana dibanding *warm-starting* penuh pada NMPC Pseudospektral Chebyshev (yang menggunakan seluruh $\mathbf{z_opt}$ dari iterasi sebelumnya sebagai tebakan).

Perhatikan urutan argumen `fmincon` berikut:

```
fmincon(cost_fun, U0, A_du, b_du, [], [], lb, ub, nonlcon, options)
```

Argumen `A_du` dan `b_du` (posisi ketiga dan keempat) adalah kendala linear ketidaksamaan $A_{\text{du}} \cdot U \leq b_{\text{du}}$ yang menyandikan batasan perubahan kemudi. Argumen `[]` pada posisi kelima dan keenam berarti tidak ada kendala linear *kesamaan*.

Jika solver gagal menemukan solusi (nilai `exitflag` ≤ 0), program memberikan peringatan dan mempertahankan tebakan awal U_0 sebagai solusi cadangan agar simulasi tidak berhenti total.

6.4. Penerapan Input Optimal dan Pembaruan State

Setelah solver menemukan rangkaian input optimal $U_{\text{opt}} = [u_1^*, u_2^*, \dots, u_N^*]^T$, hanya nilai *pertama* yang benar-benar diterapkan ke kapal. Ini adalah prinsip *Receding Horizon Control* yang menjadi landasan semua varian NMPC.

```
u_applied = U_opt(1); % Ambil nilai input optimal pertama
history_input = [history_input, u_applied]; % Menyimpan hasil input
```

6.5. Update Variabel State dengan Metode Euler

Ini adalah perbedaan arsitektural terpenting dalam NMPC konvensional. State kapal diperbarui menggunakan metode integrasi numerik Euler satu langkah ke depan.

```
% Update variabel state
dt_nd = T_sim * u_0 / L;
s_nd = euler_step(@(s,u) ship_dynamics(s, u, A_sys, B_sys, u_0_nd), s_nd, u_applied, dt_nd);
history_state_nd = [history_state_nd, s_nd]; % Menyimpan hasil state
```

Variabel dt_nd adalah langkah waktu T_{sim} yang telah dikonversi ke skala non-dimensional:

$$dt_nd = \frac{T_{\text{sim}} \cdot u_0}{L}$$

Fungsi `euler_step` mengambil argumen pertama berupa fungsi dinamika kapal yang dibungkus dalam *anonymous function* `@(s,u) ship_dynamics(s,u,...)`, memungkinkan integrasi numerik dilakukan tanpa perlu memasukkan parameter `A_sys`, `B_sys`, dan `u_0_nd` secara eksplisit setiap kali dipanggil.

Metode Euler menghitung state berikutnya dengan formula sederhana:

$$s(t + \Delta t) = s(t) + \Delta t \cdot \dot{s}(t, u)$$

di mana $\dot{s}$ adalah turunan state yang dihitung oleh fungsi `ship_dynamics`.

6.6. Pembaruan Variabel untuk Iterasi Berikutnya

```
% Update perubahan input
u_prev = u_applied;
[A_du, b_du] = du_constraints(N, u_prev, du_max);

if mod(t,20) == 0
    fprintf('t = %.1f s\n', t);
end
end
```

Setelah menerapkan input, `u_prev` diperbarui ke nilai input yang baru saja digunakan. Ini penting karena matriks kendala perubahan kemudi `A_du` dan `b_du` bergantung pada `u_prev`: batasan $|u_1 - u_{\text{prev}}| \leq \Delta u_{\text{max}}$ harus selalu mencerminkan posisi kemudi aktual saat ini. Oleh karena itu, `du_constraints` dipanggil ulang setiap iterasi untuk membangun matriks yang diperbarui.

6.7. Penyesuaian Riwayat Input dan Penghitungan Waktu

```
time_vector = 0:T_sim:T_sim_total;
% Menambahkan input terakhir untuk keperluan plotting
history_input = [history_input, history_input(end)];
waktu = toc(timer_val); % Menghentikan timer dan menghitung total waktu komputasi
fprintf('SIMULASI SELESAI\n');
fprintf('Total Waktu Komputasi: %.4f detik\n', waktu);
```

Baris `history_input = [history_input, history_input(end)]` menduplikasi nilai input terakhir dan menambahkannya ke akhir vektor. Ini dilakukan semata-mata untuk keperluan *plotting*: vektor `history_input` harus memiliki panjang yang sama dengan vektor waktu `time_vector` agar grafik dapat digambar dengan benar. Tanpa duplikasi ini, `history_input` akan memiliki satu elemen lebih sedikit dari `time_vector`.

Fungsi `toc(timer_val)` menghentikan pencatatan waktu dan mengembalikan durasi total simulasi dalam detik.

7. Konversi State ke Dimensional dan Perhitungan RMSE

```
% Konversi State ke Dimensional
hist_dim = zeros(size(history_state_nd)); % Inisialisasi matriks state dimensional
for i = 1:size(history_state_nd,2) % Loop melalui setiap kolom state dimensional
    hist_dim(:,i) = nondimensional_to_dimensional(history_state_nd(:,i), L, u_0); % Konversi ke dimensional
end
```

Semua data riwayat state yang masih berskala non-dimensional dikonversi ulang ke dalam satuan fisik (meter, m/s, radian) untuk keperluan evaluasi dan visualisasi.

```
% Perhitungan RMSE
x_ref_full = h_ref(1) + time_vector * u_0;
y_ref_full = h_ref(2) * ones(size(time_vector));
psi_ref_full = h_ref(3) * ones(size(time_vector));

rmse_x = sqrt(mean((hist_dim(3, :) - x_ref_full).^2));
rmse_y = sqrt(mean((hist_dim(4, :) - y_ref_full).^2));
rmse_psi = sqrt(mean((hist_dim(5, :) - psi_ref_full).^2));

fprintf('\n Hasil Perhitungan RMSE \n');
fprintf('RMSE X    : %.4f meter\n', rmse_x);
fprintf('RMSE Y    : %.4f meter\n', rmse_y);
fprintf('RMSE Psi   : %.4f rad (%.4f derajat)\n\n', rmse_psi, rad2deg(rmse_psi));
```

Performa akurasi dihitung menggunakan kriteria *Root Mean Square Error* (RMSE) untuk melihat seberapa besar deviasi lintasan kapal terhadap rute target ideal. Nilai RMSE yang dihasilkan di sini dapat dibandingkan langsung dengan RMSE dari NMPC Pseudospektral Chebyshev untuk mengevaluasi performa relatif kedua metode.

8. Visualisasi Hasil

```
% Plot 1: Trajektori Kapal
figure('Name', 'Lintasan Kapal');
plot(hist_dim(3,:), hist_dim(4,:), 'b-', 'LineWidth', 2); hold on;
plot(hist_dim(3,1), hist_dim(4,1), 'o', 'MarkerSize', 10, ...
    'MarkerEdgeColor', 'k', 'MarkerFaceColor', 'k');
plot(hist_dim(3,end), hist_dim(4,end), 's', 'MarkerSize', 10, ...
    'MarkerEdgeColor', 'y', 'MarkerFaceColor', 'y');
plot([0 2800], [0 0], 'r--', 'LineWidth', 1.5);
xlabel('Posisi X (m)'); ylabel('Posisi Y (m)');
title('Lintasan Kapal');
legend('NMPC', 'Titik Awal', 'Titik Akhir NMPC', 'Lintasan Referensi');
xlim([0 2800]); ylim([-100 600]);
grid on;
```

Plot lintasan menampilkan trajektori dua dimensi kapal. Titik lingkaran hitam ('o') menandai posisi awal kapal di koordinat (0,100) m, sedangkan kotak kuning ('s') menandai posisi akhir. Garis putus-putus merah menunjukkan lintasan referensi lurus $y = 0$.

```

% Plot 2: Grafik Yaw Rate
figure('Name', 'Grafik Yaw Rate');
plot(time_vector, rad2deg(hist_dim(2,:)), 'b-', 'LineWidth', 2); hold on;
yline(rad2deg(r_limit), 'r--', 'LineWidth', 1.5);
yline(-rad2deg(r_limit), 'r--', 'LineWidth', 1.5);
xlabel('Waktu (s)'); ylabel('Yaw Rate (derajat/s)');
title('Grafik Yaw Rate');
legend('NMPC', 'Batasan Yaw Rate');
grid on; ylim([-rad2deg(r_limit)*1.5, rad2deg(r_limit)*1.5]);

% Plot 3: Input u (Sudut Rudder)
figure('Name', 'Input u (Sudut Rudder)');
plot(time_vector, rad2deg(history_input), 'b-', 'LineWidth', 2); hold on;
yline(rad2deg(u_limit), 'r--', 'LineWidth', 1.5);
yline(-rad2deg(u_limit), 'r--', 'LineWidth', 1.5);
xlabel('Waktu (s)'); ylabel('Sudut Rudder (derajat)');
title('Input (u)');
legend('NMPC', 'Batasan Input');
grid on; ylim([-40 40]);

% Plot 4: Perubahan Input u
figure('Name', 'Perubahan Input u');
u_changes_norm = [0, diff(rad2deg(history_input))];
u_rate_norm = u_changes_norm / T_sim;
plot(time_vector, u_rate_norm, 'b-', 'LineWidth', 2); hold on;
yline(rad2deg(u_rate_limit), 'r--', 'LineWidth', 1.5);
yline(-rad2deg(u_rate_limit), 'r--', 'LineWidth', 1.5);
xlabel('Waktu (s)'); ylabel('Perubahan Sudut Rudder (derajat)');
title('Perubahan Input (\Delta u)');
legend('NMPC', 'Batasan Perubahan Input');
grid on; ylim([-10 10]);

```

9. Fungsi-Fungsi Pendukung

9.1. Fungsi Euler untuk Update Variabel State

```

%% Fungsi Euler
function x_next = euler_step(f, x, u, dt)
    x_next = x + dt * f(x, u);
end

```

Fungsi `euler_step` menerima empat argumen: `f` adalah fungsi dinamika kapal, `x` adalah state saat ini, `u` adalah nilai input yang diterapkan, dan `dt` adalah langkah waktu non-dimensional. Formula integrasi Euler yang diimplementasikan adalah:

$$x(t + \Delta t) \approx x(t) + \Delta t \cdot f(x(t), u(t))$$

Metode Euler mengasumsikan bahwa turunan state $\dot{x} = f(x, u)$ tetap konstan sepanjang interval $[t, t + \Delta t]$. Asumsi ini valid jika Δt cukup kecil. Dengan $T_{\text{sim}} = 1$ detik dan kapal yang bergerak relatif lambat secara rotasi, akurasi Euler masih dapat diterima meskipun bukan yang tertinggi.

9.2. Fungsi Dinamika Fisik Gerak Kapal

```

%% Fungsi Dinamika Kapal
function s_dot = ship_dynamics(s, u, A_sys, B_sys, u0_nd)
    % s = [v; r; x; y; psi]
    v = s(1); r = s(2); psi = s(5);

```

```

% Dinamika
v_r_dot = A_sys * [v; r] + B_sys * u;

% Kinematika
x_dot = u0_nd*cos(psi) - v*sin(psi);
y_dot = u0_nd*sin(psi) + v*cos(psi);

s_dot = [v_r_dot(1); v_r_dot(2); x_dot; y_dot; r];
end

```

Fungsi menerima vektor state $s = [v, r, x, y, \psi]^T$ dan nilai input u (sudut kemudi), lalu mengembalikan vektor turunan $\dot{s} = [\dot{v}, \dot{r}, \dot{x}, \dot{y}, \dot{\psi}]^T$. Dinamika kecepatan lateral dan putar $[\dot{v}, \dot{r}]$ dihitung dari persamaan sistem linear $A_{\text{sys}}[v; r] + B_{\text{sys}}u$, sedangkan kinematika posisi $[\dot{x}, \dot{y}, \dot{\psi}]$ dihitung dari transformasi koordinat badan ke koordinat dunia.

9.3. Fungsi Biaya NMPC (Pendekatan Sequential)

Pada fungsi ini, state kapal dihitung satu per satu secara berurutan (*sequential*) dari kondisi awal menggunakan Euler.

```

% Fungsi Biaya NMPC (Sequential Approach)
function J = mpc_cost(U, s0, u_prev, x_ref_seq, y_ref_seq, psi_ref_seq, ...
                    T_sim, L, u0, A_sys, B_sys, u0_nd, Q, R)
    N = length(U); % langkah dalam horizon prediksi
    s = s0;
    u_last = u_prev;
    dt_nd = T_sim * u0 / L;
    J = 0; % Inisialisasi nilai objektif

    for i = 1:N
        u = U(i);
        % Prediksi state dengan metode Euler
        s = euler_step(@(s,u) ship_dynamics(s, u, A_sys, B_sys, u0_nd), s, u, dt_nd);

        % Error pada (x, y, psi) nondimensional
        err = [s(3) - x_ref_seq(i);
                s(4) - y_ref_seq(i);
                s(5) - psi_ref_seq(i)];

        J = J + err' * Q * err + R * u^2;
        u_last = u;
    end
end

```

Fungsi ini berjalan sebagai berikut: dimulai dari kondisi awal s_0 (posisi kapal saat ini), loop maju satu langkah Euler menggunakan u_i dari vektor U , menghasilkan prediksi state s pada waktu $t + i \cdot T_{\text{sim}}$. Error antara state prediksi dan lintasan referensi kemudian dihitung dan dijumlahkan ke nilai biaya J .

9.4. Fungsi Kendala Nonlinear: Batasan Yaw Rate

```

% Fungsi Kendala Nonlinear: Batasan Yaw Rate
function [c, ceq] = state_constraints(U, s0, T_sim, L, u0, A_sys, B_sys, u0_nd, r_limit_nd)
    N = length(U);
    s = s0;
    dt_nd = T_sim * u0 / L;
    c = [];

    for i = 1:N
        % Prediksi state

```



```

    s = euler_step(@(s,u) ship_dynamics(s, u, A_sys, B_sys, u0_nd), s, U(i), dt_nd);
    r = s(2);    % yaw rate nondimensional

    % Menambahkan kendala yaw rate
    c = [c; r - r_limit_nd; -r_limit_nd - r];
end
ceq = [];
end

```

Fungsi ini menghasilkan keluaran c (kendala pertidaksamaan) dan ceq (kendala kesamaan). Nilai $ceq = []$ berarti tidak ada kendala kesamaan nonlinear.

Di dalam loop, state kapal dipropagasi maju langkah demi langkah menggunakan Euler (sama persis seperti di fungsi biaya). Untuk setiap langkah i , dua kendala *yaw rate* didaftarkan ke vektor c :

$$r - r_{\text{limit}} \leq 0 \quad \Leftrightarrow \quad r \leq r_{\text{limit}}$$

$$-r_{\text{limit}} - r \leq 0 \quad \Leftrightarrow \quad r \geq -r_{\text{limit}}$$

Kedua baris ini bersama-sama memastikan $|r| \leq r_{\text{limit}}$ di setiap langkah waktu. Teknik ini menghasilkan $2N = 60$ kendala nonlinear untuk $N = 30$ langkah horizon.

9.5. Fungsi Pembentukan Kendala Linear Perubahan Input

Fungsi ini membangun matriks kendala linear A_{du} dan vektor b_{du} yang menyandikan batasan perubahan sudut kemudi antar langkah waktu berturutan.

```

% Fungsi Pembuatan Kendala Linear untuk Perubahan Input (du)
function [A, b] = du_constraints(N, u_prev, du_max)
    A = zeros(2*N, N);
    b = zeros(2*N, 1);

    % u0 - u_prev <= du_max
    A(1,1) = 1;    b(1) = u_prev + du_max;
    % u_{k} - u_{k-1} <= du_max
    for i = 2:N
        A(i, i-1) = -1;    A(i, i) = 1;
        b(i) = du_max;
    end

    % -u0 + u_prev <= du_max
    A(N+1,1) = -1;    b(N+1) = du_max - u_prev;
    % -u_k + u_{k-1} <= du_max
    for i = 2:N
        A(N+i, i-1) = 1;    A(N+i, i) = -1;
        b(N+i) = du_max;
    end
end
end

```

Fungsi ini menghasilkan matriks A berukuran $2N \times N$ dan vektor b berukuran $2N \times 1$. Setengah bagian atas berisi kendala bahwa kemudi tidak boleh bergerak *terlalu cepat ke kanan* ($u_k - u_{k-1} \leq \Delta u_{\text{max}}$), dan setengah bagian bawah berisi kendala bahwa kemudi tidak boleh bergerak *terlalu cepat ke kiri* ($u_{k-1} - u_k \leq \Delta u_{\text{max}}$). Keduanya bersama-sama memastikan $|u_k - u_{k-1}| \leq \Delta u_{\text{max}}$.

Baris pertama ($i = 1$) berbeda dari baris lainnya karena melibatkan u_{prev} (nilai kemudi aktual sebelum horizon), bukan kemudi sesama variabel keputusan. Persamaan $u_1 - u_{\text{prev}} \leq \Delta u_{\text{max}}$ diubah ke bentuk $u_1 \leq u_{\text{prev}} + \Delta u_{\text{max}}$, sehingga cukup $A(1,1) = 1$ dan $b(1) = u_{\text{prev}} + \Delta u_{\text{max}}$.

Sebagai contoh sederhana untuk $N = 3$, matriks A_{du} dan vektor b_{du} yang dihasilkan adalah:

$$A_{\text{du}} = \begin{bmatrix} 1 & 0 & 0 \\ -1 & 1 & 0 \\ 0 & -1 & 1 \\ -1 & 0 & 0 \\ 1 & -1 & 0 \\ 0 & 1 & -1 \end{bmatrix}, \quad b_{\text{du}} = \begin{bmatrix} u_{\text{prev}} + \Delta u_{\text{max}} \\ \Delta u_{\text{max}} \\ \Delta u_{\text{max}} \\ \Delta u_{\text{max}} - u_{\text{prev}} \\ \Delta u_{\text{max}} \\ \Delta u_{\text{max}} \end{bmatrix}$$

9.6. Fungsi Konversi Non-dimensional dan Dimensional

```
function x_nd = dimensional_to_nondimensional(x, L, u0)
    x_nd = [x(1)/u0; x(2)*L/u0; x(3)/L; x(4)/L; x(5)];
end

function x_dim = nondimensional_to_dimensional(x, L, u0)
    x_dim = [x(1)*u0; x(2)*u0/L; x(3)*L; x(4)/L; x(5)];
end
```